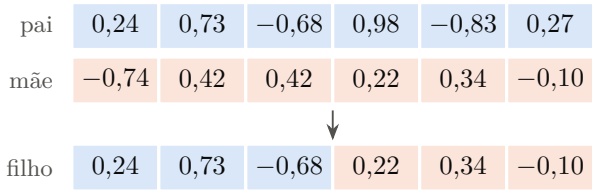
Algoritmo Genético para Classificação

Como o algoritmo funciona e como usar a biblioteca



Fellipe Almassy · **Matheus Gomes**

Novembro de 2026

Sumário

Sobre este projeto 2

I Como funciona o algoritmo genético 4

Objetivo 4

Etapa 1: criação da população 5

Etapa 2: cálculo do fitness 7

Etapa 3: seleção dos pais 10

Etapa 4: cruzamento 12

Etapa 5: mutação 15

Etapa 6: atualização da população 18

II Como usar a biblioteca 21

Os quatro passos 21

Instalação 21

O que entra e o que sai de cada função 22

1) Carregar os dados 23

2) Treinar 24

3) Prever 25

4) Avaliar 26

Classificar dados novos 28

Com mais de duas classes 30

Parâmetros do `treinar` 32

Usando cada etapa separadamente 33

Erros comuns 34

Sobre este projeto

Somos dois estudantes de matemática aplicada e computação, e decidimos transformar em biblioteca um conteúdo que vimos em aula: o algoritmo genético aplicado à classificação.

Por que fizemos do zero

O objetivo deste projeto não é criar uma superbiblioteca para desbancar o scikit-learn, até porque isso seria impossível. Sentimos a necessidade de entender os modelos com mais profundidade. Usar somente bibliotecas prontas dá um resultado muito imediato, sem exigir muito entendimento, e por isso resolvemos montar uma do zero.

Combinamos uma regra desde o início: usar apenas bibliotecas de base, como o NumPy e o pandas, e fazer todo o resto na mão. A criação da população, o fitness, a seleção, o cruzamento, a mutação e a atualização foram escritos por nós, sem nenhuma biblioteca pronta de aprendizado de máquina ou de algoritmos genéticos.

Como o projeto evoluiu

A primeira versão foi feita 100% na mão e sem inteligência artificial. Ela funcionava, mas não era escalável. Ao longo de 3 meses, fomos generalizando o código e entendendo melhor o problema, até chegar à biblioteca descrita neste documento.

Como usamos inteligência artificial

Usamos inteligência artificial nesse caminho, principalmente nos detalhes e para melhorar a experiência de uso. Ela nos ajudou a:

- testar erros;
- rodar benchmarks;
- usar funções que não conhecíamos dentro do NumPy e do pandas.

Onde usamos o algoritmo

Todos estes exemplos, e outros que ainda estão por vir, serão disponibilizados na seção de exemplos do repositório no GitHub.

Problema	O que o modelo classifica	Situação
Clientes de um banco	adimplentes ou inadimplentes	usado em projeto; será publicado como notebook didático
Cogumelos	comestíveis ou não comestíveis	usado em projeto
Resultados de exames	normal, anormal ou inconclusivo	em desenvolvimento

Por razões de privacidade, não podemos citar o nome do banco.

O exemplo que estamos preparando: dados de saúde

Para nós, este é o exemplo mais interessante. Ele usa o Healthcare Dataset, publicado no Kaggle, que simula registros de internações hospitalares. Cada linha descreve um paciente, com dados como idade, gênero, tipo sanguíneo, condição médica, tipo de admissão, medicamento, plano de

saúde e valor cobrado. O objetivo é prever a coluna *Test Results*, o resultado de um exame, que pode ser *Normal*, *Abnormal* ou *Inconclusive*.

Esse problema traz três desafios que os outros exemplos não têm juntos:

- **Três classes:** usa o modelo ordinal, então será preciso definir uma ordem entre os resultados. Com 3 classes, são 3 ordens diferentes a testar, pois cada ordem e a sua inversa dão o mesmo resultado.
- **Dados em texto:** a biblioteca só aceita features numéricas, então colunas como condição médica e medicamento precisarão ser convertidas em números.
- **Dados sintéticos:** a base foi gerada artificialmente, sem nenhum paciente real, o que permite estudar saúde sem expor dados sensíveis. Em contrapartida, pode não haver uma relação real entre as features e o resultado do exame, e descobrir isso faz parte do estudo.

De onde veio este documento

Este documento é uma junção de materiais das aulas que tivemos, de rascunhos e de anotações matemáticas nossas. Todo esse material foi passado para a inteligência artificial, e o resultado foi verificado por nós.

Agradecimento especial

Agradecemos especialmente ao professor Aguinaldo Prandini Ricieri, nosso professor de matemática aplicada, por nos apresentar ao conceito de algoritmo genético, por nos fornecer bases de dados reais e por todas as aulas incríveis que só encontramos no Prandiano.

Segundo o site do Prandiano, o professor Ricieri é bacharel em Física pela USP, pós-graduado em Engenharia Espacial pelo ITA, onde se tornou professor em 1982, consultor, autor de vários livros e curador do Museu de Matemática Prandiano. O site também descreve consultorias prestadas a empresas como Petrobras, Vale e Pirelli, que ele leva para a sala de aula como exemplos reais de matemática aplicada.

Temos aula com ele no próprio Museu de Matemática Prandiano, em São Paulo.

O que vem a seguir

O resultado desses 3 meses de dedicação é a nossa primeira biblioteca de aprendizado de máquina. Ainda há melhorias por vir, e feedbacks são muito bem-vindos.

Contato

Nome	LinkedIn	Email	GitHub
Fellipe Almassy	link do LinkedIn	fealmassy@gmail.com	FeAlmassy
Matheus Gomes	link do LinkedIn	matheusgomesabreu2006@gmail.com	matheos-hash

O código está em github.com/FeAlmassy/AlgoritmoGenetico.

Parte I

Como funciona o algoritmo genético

Objetivo

O objetivo do algoritmo genético é encontrar os coeficientes de uma função de decisão que classifique corretamente as lições de uma base de dados, a partir de suas features. Depois do treinamento, essa função pode ser usada para classificar dados novos.

Para cada lição, a função calcula uma nota:

$$\text{nota} = \text{bias} + \text{gene}_1 \cdot x_1 + \text{gene}_2 \cdot x_2 + \cdots + \text{gene}_n \cdot x_n$$

No caso de duas classes, a nota define a classe prevista:

$$\text{nota} \geq 0 \;\Rightarrow\; \text{classe } 1 \qquad\qquad \text{nota} < 0 \;\Rightarrow\; \text{classe } 0$$

Em vez de calcular esses coeficientes diretamente, o algoritmo os faz evoluir. A Figura 1 mostra o ciclo completo, e cada etapa é explicada nas próximas seções.

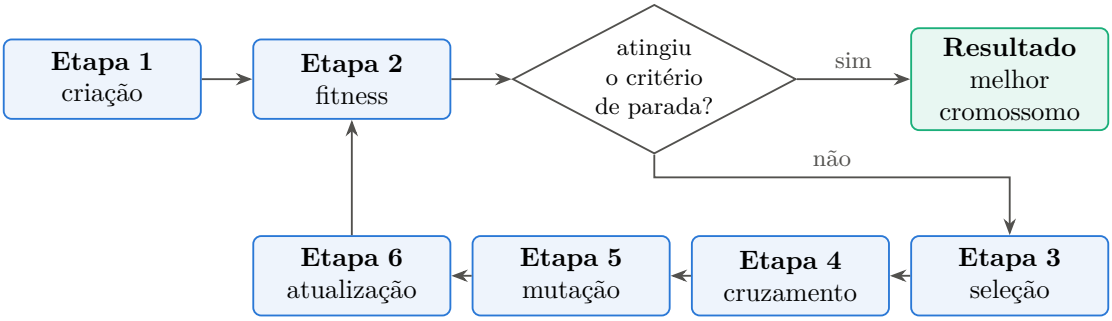


Figura 1. O ciclo do algoritmo genético. Cada volta é uma geração.

O vocabulário do algoritmo

O algoritmo se inspira na evolução das espécies e usa os nomes dela:

Termo	O que significa neste algoritmo
Gene	um coeficiente da função de decisão
Cromossomo	uma solução candidata: o bias e um peso para cada feature
População	o conjunto de cromossomos que existe em um dado momento
Fitness	a nota de um cromossomo: o quanto ele acerta todas as classes
Filho	um cromossomo novo, gerado a partir de dois cromossomos da população
Geração	uma volta completa no ciclo da Figura 1

Por que usar um algoritmo genético

Métodos baseados em gradiente ajustam os coeficientes seguindo a direção em que a nota melhora, o que exige uma nota que mude suavemente. O fitness deste algoritmo só muda quando alguma lição troca de classe, então uma pequena mudança em um gene pode não alterar nada. Como o algoritmo genético só precisa comparar notas, ele consegue usar esse fitness diretamente.

Etapa 1: criação da população

O programa começa criando uma população de cromossomos. Cada cromossomo é um vetor com uma solução candidata, ou seja, um conjunto de coeficientes:

$$
\begin{aligned}
C_1 &= [\text{bias},\ \text{gene}_1,\ \text{gene}_2,\ \ldots,\ \text{gene}_n] \\
C_2 &= [\text{bias},\ \text{gene}_1,\ \text{gene}_2,\ \ldots,\ \text{gene}_n] \\
&\vdots \\
C_m &= [\text{bias},\ \text{gene}_1,\ \text{gene}_2,\ \ldots,\ \text{gene}_n]
\end{aligned}
$$

Cada gene é o peso de uma feature, então um cromossomo tem um gene para cada feature, mais o bias.

O bias é o termo independente da função e desloca a fronteira de decisão. Sem ele, a fronteira passaria sempre pela origem, e uma lição com todas as features iguais a 0 teria sempre nota 0, quaisquer que fossem os pesos. A Figura 2 mostra um caso em que ele faz diferença.

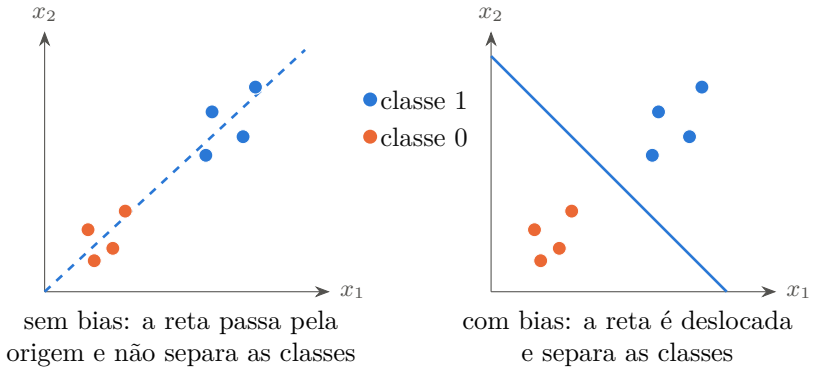


Figura 2. O efeito do bias na fronteira de decisão, em um exemplo com duas features.

Todos os valores iniciais, incluindo o bias, são sorteados dentro de um intervalo. O padrão é de -1 a 1, e pode ser alterado via parâmetro.

O resultado desta etapa é uma matriz em que cada linha é um cromossomo e cada coluna é um gene (Figura 3).

	bias	gene_1	gene_2	gene_3	gene_4
C_1	0,24	0,73	−0,68	0,98	−0,83
C_2	−0,74	0,42	0,42	0,22	0,34
C_3	0,61	−0,15	0,87	−0,42	0,09
C_4	−0,33	0,56	−0,91	0,18	0,77

Figura 3. Uma população de 4 cromossomos para uma base com 4 features.

Por que começar ao acaso

O sorteio espalha a busca por soluções diferentes, sem depender de nenhum palpite inicial. Para ver o ponto de partida, sorteamos 1000 cromossomos para os dados de treino da base `dados_binario_1.xlsx`:

Medida	Valor
Fitness médio	0,171
Melhor fitness	0,7136
Cromossomos com fitness 0	7,7%

Mesmo sem treino, alguns cromossomos já são razoáveis, e a maioria é ruim. As próximas etapas partem dessa variedade.

Quantos cromossomos criar

O tamanho da população é um parâmetro, e o valor padrão da biblioteca é 6. Em nossos testes, com 10 sementes, 500 gerações, fitness alvo de 1,0 e os valores padrão nos demais parâmetros, com a base `dados_binario_1.xlsx`, ele fez pouca diferença:

Cromossomos	Melhor fitness no treino (média)	Acurácia no teste (média)
3	0,8062	81,8%
6	0,8123	82,6%
12	0,8151	82,4%
24	0,8099	83,5%
48	0,8044	83,1%

Essas diferenças são pequenas e podem mudar com outras sementes ou bases, então a tabela não aponta um tamanho ideal. Como só um casal se reproduz a cada geração nesta biblioteca, uma população maior também é renovada mais devagar.

Etapa 2: cálculo do fitness

Nesta etapa, cada cromossomo recebe uma nota de qualidade, chamada fitness. Um bom cromossomo é aquele que acerta bem as classes da base de dados.

Exemplo

Considere a seguinte base:

	X1	X2	X3	X4	Classe
Lição 1	1	0	1	1	1
Lição 2	1	1	0	0	1
Lição 3	1	0	0	1	0
Lição 4	0	1	1	1	0

E o seguinte cromossomo, com valores inteiros escolhidos apenas para facilitar as contas (por isso, alguns ficam fora do intervalo padrão de -1 a 1):

$$C_1 = [2,\ 3,\ -4,\ 1,\ -1]$$

Aplicando esse cromossomo a cada lição:

Lição	Cálculo	Nota	Prevista	Real	Resultado
1	$2+3\cdot 1-4\cdot 0+1\cdot 1-1\cdot 1$	5	1	1	acertou
2	$2+3\cdot 1-4\cdot 1+1\cdot 0-1\cdot 0$	1	1	1	acertou
3	$2+3\cdot 1-4\cdot 0+1\cdot 0-1\cdot 1$	4	1	0	**errou**
4	$2+3\cdot 0-4\cdot 1+1\cdot 1-1\cdot 1$	-2	0	0	acertou

O cromossomo acertou 3 das 4 lições, uma acurácia de 75%.

Por que não usar a acurácia como nota

Bases de dados costumam ser desbalanceadas (ter muito mais lições de algumas classes do que de outras), e nesse caso a acurácia pode premiar um cromossomo inútil. Em uma base com 99 lições da classe 1 e apenas 1 da classe 0, um cromossomo que responde sempre "classe 1" tem 99% de acurácia sem ter aprendido a separar as classes.

Por isso, o fitness mede o percentual de acerto dentro de cada classe (o *recall*) e multiplica os resultados:

$$\mathrm{Pa}_{\text{classe }0} = \frac{\text{acertos na classe 0}}{\text{total da classe 0}} \qquad \mathrm{Pa}_{\text{classe }1} = \frac{\text{acertos na classe 1}}{\text{total da classe 1}}$$

$$\text{Fitness} = \mathrm{Pa}_{\text{classe }0} \times \mathrm{Pa}_{\text{classe }1}$$

No exemplo anterior:

$$\mathrm{Pa}_{\text{classe }1} = \tfrac{2}{2} = 1{,}0 \qquad \mathrm{Pa}_{\text{classe }0} = \tfrac{1}{2} = 0{,}5 \qquad \text{Fitness} = 1{,}0 \times 0{,}5 = 0{,}5$$

A acurácia foi de 0,75, mas o fitness foi de 0,5, porque o cromossomo acertou apenas metade da classe 0.

Por que multiplicar em vez de tirar a média

Voltando à base com 99 lições da classe 1 e 1 da classe 0, o cromossomo que responde sempre "classe 1" tem as notas da Figura 4.

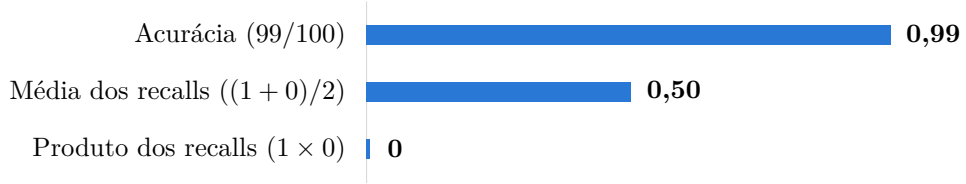


Figura 4. A nota de um cromossomo que responde sempre "classe 1", em três formas de calcular.

A média ainda dá uma nota razoável a quem ignorou uma classe inteira. Já o produto zera o fitness, então um cromossomo só recebe nota alta se for bem em todas as classes. Por isso, usamos como nota de cada cromossomo o produto dos percentuais de acerto por classe, ou seja, o produto dos recalls.

Generalização para várias classes

Com K classes, o fitness é o produto dos K recalls:

$$\text{Fitness} = \text{Pa}_{\text{classe }0} \times \text{Pa}_{\text{classe }1} \times \cdots \times \text{Pa}_{\text{classe }K-1}$$

Como a nota continua sendo um único número, ela precisa ser dividida em K faixas, uma para cada classe. Essa divisão é feita por $K - 1$ pontos de corte em ordem crescente, chamados *thresholds*. Com 5 classes, por exemplo, são 4 cortes (Figura 5).

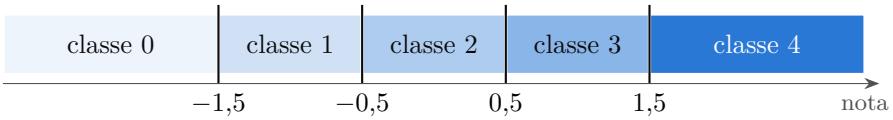


Figura 5. Os thresholds dividem a nota em uma faixa para cada classe.

Uma nota igual a um corte fica na faixa de cima, e o caso binário é o caso particular com um único corte em 0.

Como as faixas ficam em sequência na reta, as classes precisam ter uma ordem, da menor para a maior (por exemplo, ruim < médio < bom). Por isso, este modelo é chamado de *ordinal*.

O valor do fitness também tende a cair quando o número de classes aumenta, pois mais fatores menores que 1 são multiplicados. Com 5 classes e 80% de acerto em cada uma, o fitness é $0{,}8^5 \approx 0{,}33$, embora o modelo acerte 80% das lições. Por isso, um mesmo valor de fitness não tem o mesmo significado em problemas com números diferentes de classes.

> Ao rodar o modelo, é importante olhar tanto o fitness quanto a acurácia: a acurácia é fácil de interpretar, mas pode enganar em bases desbalanceadas, enquanto o fitness mostra se o cromossomo vai bem em todas as classes. Como as duas medidas retornadas pelo treino são calculadas nos próprios dados de treino, o desempenho em dados novos deve ser avaliado em um conjunto de teste, separado com `dividir_treino_teste`, como veremos na Parte II.

Fitness maior nem sempre significa acurácia maior

Como as duas medidas avaliam coisas diferentes, elas podem discordar. Em uma base com 90 lições da classe 1 e 10 da classe 0:

Cromossomo	Recall da classe 1	Recall da classe 0	Fitness	Acurácia
A	0,9	0,5	0,45	86%
B	0,7	0,7	0,49	70%

O cromossomo B tem o fitness maior porque equilibra melhor as classes, mas A acerta mais lições no total. O algoritmo prefere B, e qual deles é melhor depende do que importa no problema.

Como ler o valor do fitness

O fitness vai de 0 a 1, e o valor depende do número de classes. A tabela mostra o fitness de modelos com o mesmo recall em todas as classes:

Classes	Recall de 90%	Recall de 80%	Recall de 70%
2	0,81	0,64	0,49
3	0,73	0,51	0,34
5	0,59	0,33	0,17

O fitness em dados reais

Estes são os resultados no conjunto de teste das duas bases de exemplo, que aparecem na Parte II:

Base	Classes	Recall por classe	Fitness	Acurácia
`dados_binario_1.xlsx`	2	85%, 79%	0,6648	81,5%
`dados_multiclasse.xlsx`	5	46%, 100%, 75%, 75%, 92%	0,2380	75,0%

A acurácia das duas bases é parecida, mas o fitness é bem menor na base com 5 classes, tanto pelo número de fatores quanto pelo recall de 46% em uma delas.

Etapa 3: seleção dos pais

Depois que todos os cromossomos recebem um fitness, o algoritmo escolhe quais deles vão se reproduzir. A ideia vem da seleção natural: os indivíduos mais adaptados têm mais chance de passar seus genes adiante.

Essa escolha é um sorteio, não uma simples escolha dos melhores. Um cromossomo com fitness alto tem mais chance de ser selecionado, mas nada garante que ele será escolhido. Um dos métodos mais usados para fazer esse sorteio é a roleta.

Como funciona a roleta

Imagine uma roleta em que cada cromossomo ocupa uma fatia. Quanto maior o fitness, maior a fatia e maior a chance de a roleta parar nela. A chance de cada cromossomo é a sua parte no fitness total da população:

$$\text{chance do cromossomo} = \frac{\text{fitness do cromossomo}}{\text{soma dos fitnesses da população}}$$

Exemplo

Considere uma população de 4 cromossomos, cuja soma dos fitnesses é 0,80:

Cromossomo	Fitness	Cálculo	Chance	Faixa do sorteio
C_1	0,40	0,40 / 0,80	50%	de 0 a 50
C_2	0,24	0,24 / 0,80	30%	de 50 a 80
C_3	0,12	0,12 / 0,80	15%	de 80 a 95
C_4	0,04	0,04 / 0,80	5%	de 95 a 100

Para girar a roleta, sorteamos um número entre 0 e 100. Cada cromossomo fica com uma faixa do tamanho da sua chance, e é selecionado quando o número cai nela (Figura 6). Como são necessários dois pais, a roleta é girada duas vezes:

- 1º giro: saiu 62, que cai na faixa de C_2, então C_2 é o pai.
- 2º giro: saiu 31, que cai na faixa de C_1, então C_1 é a mãe.

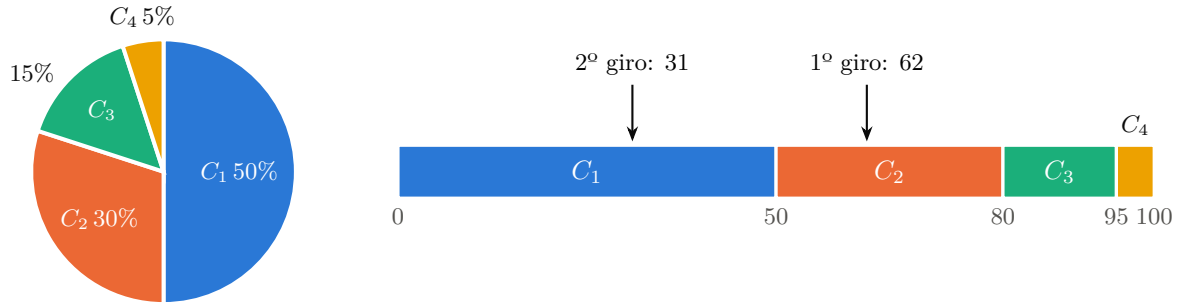


Figura 6. A mesma roleta vista como círculo e como faixa de 0 a 100.

Como os giros são independentes, o mesmo cromossomo pode ser sorteado duas vezes. Nesse caso, ele é pai e mãe ao mesmo tempo, e os filhos saem iguais a ele antes da mutação.

Por que não escolher sempre os melhores

Pode parecer mais eficiente cruzar sempre os dois melhores cromossomos, mas isso faria a população perder diversidade rapidamente. Os cromossomos ficariam cada vez mais parecidos, e o algoritmo poderia ficar preso em uma solução apenas razoável, sem explorar outras possibilidades.

Um cromossomo com fitness baixo também pode ter genes úteis, que só mostram seu valor quando combinados com os genes de outro cromossomo. A roleta mantém essa possibilidade aberta. No exemplo, C_4 tem só 5% de chance por giro, mas ainda pode ser escolhido.

Por que não sortear com chances iguais

No outro extremo, se todos tivessem a mesma chance, o fitness não influenciaria a reprodução. Os genes bons não seriam favorecidos, e a busca perderia a direção, parecendo mais uma tentativa ao acaso.

A roleta fica entre esses dois extremos:

Forma de escolher	Vantagem	Problema
Sempre os melhores	Favorece os bons genes	Perde diversidade
Chances iguais	Mantém a diversidade	Não favorece os bons genes
Roleta	Favorece os bons genes e mantém a diversidade	Continua dependendo do acaso

Com o pai e a mãe escolhidos, o algoritmo passa para a próxima etapa: o cruzamento, em que os genes dos dois são combinados para formar os filhos.

A chance real de cada cromossomo

Como a roleta gira duas vezes, a chance de um cromossomo ser escolhido ao menos uma vez em uma geração é maior que a chance por giro:

$$\text{chance em dois giros} = 1 - (1 - \text{chance por giro})^2$$

Cromossomo	Chance por giro	Chance de ser escolhido em dois giros
C_1	50%	75,0%
C_2	30%	51,0%
C_3	15%	27,8%
C_4	5%	9,8%

Já a chance de o mesmo cromossomo ser sorteado nos dois giros é a soma das chances ao quadrado:

$$0{,}50^2 + 0{,}30^2 + 0{,}15^2 + 0{,}05^2 = 0{,}365$$

Nessa população, isso acontece em 36,5% das gerações. Por isso, a mutação é importante mesmo quando o casal é formado por um único cromossomo.

Etapa 4: cruzamento

No cruzamento, os genes do pai e da mãe são combinados para formar novos cromossomos, os filhos. A ideia é que dois cromossomos razoáveis podem ter acertado partes diferentes do problema, e um filho pode herdar o que cada um tem de melhor.

Como funciona o corte em um ponto

Um dos métodos mais simples é o cruzamento de um ponto:

1. Sorteamos um ponto de corte entre dois genes.
2. O filho recebe os genes do pai até o corte.
3. Depois do corte, ele recebe os genes da mãe.

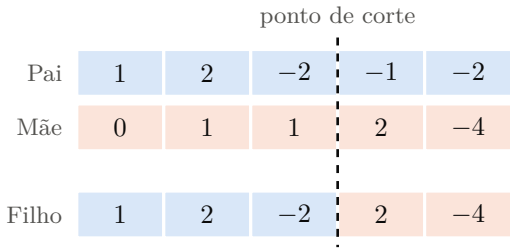


Figura 7. O filho herda o começo do pai e o final da mãe.

O corte fica sempre entre dois genes, nunca antes do primeiro ou depois do último. Assim, o filho sempre herda genes dos dois pais, em vez de ser uma cópia de apenas um deles. Um cromossomo com n genes tem $n - 1$ pontos de corte possíveis.

Cada filho tem seu próprio corte sorteado. Por isso, um mesmo casal pode gerar filhos diferentes.

Exemplo

Vamos usar a mesma base da Etapa 2:

	X1	X2	X3	X4	Classe
Lição 1	1	0	1	1	1
Lição 2	1	1	0	0	1
Lição 3	1	0	0	1	0
Lição 4	0	1	1	1	0

Os dois pais têm o mesmo fitness, mas erram lições diferentes:

Cromossomo	Genes	Lição que erra	Fitness
Pai	[1, 2, −2, −1, −2]	Lição 3	0,5
Mãe	[0, 1, 1, 2, −4]	Lição 1	0,5

Com 5 genes, existem 4 pontos de corte possíveis. Estes são os filhos que cada um deles geraria:

Corte depois do	Filho	Lição que erra	Fitness
1º gene	$[1,\ 1,\ 1,\ 2,\ -4]$	Lição 4	0,5
2º gene	$[1,\ 2,\ 1,\ 2,\ -4]$	Lição 4	0,5
3º gene	$[1,\ 2,\ -2,\ 2,\ -4]$	nenhuma	**1,0**
4º gene	$[1,\ 2,\ -2,\ -1,\ -4]$	Lição 1	0,5

Veja como o filho do corte depois do 3º gene se sai em cada lição:

Lição	Cálculo	Nota	Prevista	Real	Resultado
1	$1+2\cdot 1-2\cdot 0+2\cdot 1-4\cdot 1$	1	1	1	acertou
2	$1+2\cdot 1-2\cdot 1+2\cdot 0-4\cdot 0$	1	1	1	acertou
3	$1+2\cdot 1-2\cdot 0+2\cdot 0-4\cdot 1$	-1	0	0	acertou
4	$1+2\cdot 0-2\cdot 1+2\cdot 1-4\cdot 1$	-3	0	0	acertou

Nenhum dos pais acertava todas as lições, mas essa combinação acerta, e o fitness sobe de 0,5 para 1,0.

Por que o cruzamento funciona

O cruzamento aproveita soluções que já foram testadas. Um gene isolado não é bom nem ruim, porque a nota depende da soma de todos os genes. Por isso, a mesma parte de um cromossomo pode funcionar bem ou mal dependendo dos genes que a acompanham, e cruzar é uma forma de testar novas combinações.

Isso não garante um filho melhor. No exemplo, três dos quatro filhos possíveis ficaram com o mesmo fitness dos pais, e em outros casos um filho pode até ficar pior. O cruzamento só aumenta as chances de encontrar boas combinações, e são as próximas etapas que decidem o que permanece na população.

O limite do cruzamento

O cruzamento só reorganiza valores que já existem nos pais, sem criar nenhum valor novo. No exemplo, o primeiro gene de qualquer filho sempre será 1, porque o corte nunca acontece antes dele e o filho sempre começa com os genes do pai. Se um valor necessário não estiver em nenhum dos dois pais, o cruzamento não consegue produzi-lo.

É esse limite que a próxima etapa resolve: a mutação, que introduz valores novos nos filhos.

Quantos filhos gerar

A quantidade de filhos por geração é um parâmetro, e o valor padrão da biblioteca é 3. Em nossos testes, com 10 sementes, 500 gerações, fitness alvo de 1,0 e os valores padrão nos demais parâmetros, com a base `dados_binario_1.xlsx`, ela fez pouca diferença:

Filhos	Melhor fitness no treino (média)	Acurácia no teste (média)
2	0,8155	84,3%
3	0,8123	82,6%
6	0,8149	83,2%
12	0,8137	83,4%

Outras formas de cruzar

O corte em um ponto é o método usado nesta biblioteca, mas não é o único:

Método	Como funciona
Um ponto	o filho pega o começo de um dos pais e o final do outro
Dois pontos	o filho pega o trecho entre dois cortes de um dos pais, e o resto do outro
Uniforme	cada gene do filho é sorteado, vindo do pai ou da mãe

Quantos cortes existem em bases reais

Cada ponto de corte tem a mesma chance de ser sorteado, então a chance de um corte específico é de $1/(n-1)$, em que n é o número de genes:

Base	Genes	Pontos de corte	Chance de cada corte
Exemplo deste documento	5	4	25%
`dados_binario_1.xlsx`	19	18	5,6%
`dados_multiclasse.xlsx`	31	30	3,3%

Um cruzamento em dados reais

No notebook `exemplo_binario.ipynb`, a primeira geração escolheu pais com fitness de 0,1498 e 0,0827. Depois do cruzamento e da mutação, os três filhos ficaram com fitness de 0,4515, 0,1933 e 0,2531, todos acima dos dois pais.

Em uma única geração, o melhor fitness da população subiu de 0,2204 para 0,4515. Esse salto é comum no início, quando os cromossomos ainda são ruins, e fica mais raro com o tempo, como mostra a Figura 10.

Etapa 5: mutação

Na mutação, alguns genes de cada filho são trocados por valores novos, sorteados ao acaso. Assim como na natureza, essa mudança não tem direção: ela pode ajudar, atrapalhar ou não fazer diferença.

Sem a mutação, o algoritmo só poderia recombinar os valores que já existem na população. A mutação é o que traz valores novos para a busca.

Como funciona a mutação

Para cada filho gerado no cruzamento:

1. Sorteamos quais genes serão alterados. A quantidade é definida por um parâmetro.
2. Trocamos cada gene sorteado por um novo valor aleatório entre -1 e 1.

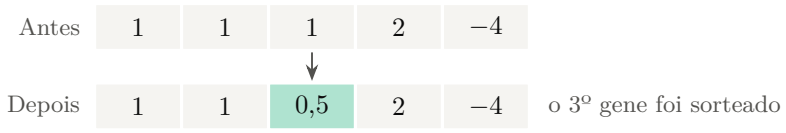


Figura 8. Uma mutação troca o valor do gene sorteado por um valor novo.

O novo valor não depende do anterior. O gene é substituído, e não ajustado aos poucos.

Exemplo

Vamos usar novamente a base da Etapa 2:

	X1	X2	X3	X4	Classe
Lição 1	1	0	1	1	1
Lição 2	1	1	0	0	1
Lição 3	1	0	0	1	0
Lição 4	0	1	1	1	0

Na Etapa 4, o filho $[1,\ 1,\ 1,\ 2,\ -4]$ errou a Lição 4 e ficou com fitness 0,5. Veja três mutações possíveis nesse filho, cada uma com um efeito diferente (nesta tabela, os genes são separados por ponto e vírgula, porque 0,5 é um único valor):

Gene alterado	Cromossomo após a mutação	Lições que erra	Fitness	Efeito
3º gene: de 1 para 0,5	$[1;\ 1;\ 0{,}5;\ 2;\ -4]$	nenhuma	**1,0**	melhorou
1º gene: de 1 para -1	$[-1;\ 1;\ 1;\ 2;\ -4]$	Lição 1	0,5	manteve o fitness
5º gene: de -4 para 1	$[1;\ 1;\ 1;\ 2;\ 1]$	Lições 3 e 4	0	piorou

Veja como o cromossomo que melhorou se sai em cada lição:

Lição	Cálculo	Nota	Prevista	Real	Resultado
1	$1+1\cdot 1+0{,}5\cdot 0+2\cdot 1-4\cdot 1$	0	1	1	acertou
2	$1+1\cdot 1+0{,}5\cdot 1+2\cdot 0-4\cdot 0$	2,5	1	1	acertou
3	$1+1\cdot 1+0{,}5\cdot 0+2\cdot 0-4\cdot 1$	-2	0	0	acertou
4	$1+1\cdot 0+0{,}5\cdot 1+2\cdot 1-4\cdot 1$	$-0{,}5$	0	0	acertou

O valor 0,5 não existia em nenhum dos pais, então o cruzamento nunca chegaria a esse cromossomo. Foi a mutação que tornou essa solução possível.

Por que a mutação é necessária

- **Traz valores novos:** o cruzamento só combina o que já existe. A mutação alcança valores que nenhum cromossomo da população tinha.
- **Recupera o que se perdeu:** um valor útil pode desaparecer quando os cromossomos que o carregam saem da população. A mutação é uma forma de voltar a testar valores diferentes.
- **Ajuda a sair de soluções apenas razoáveis:** quando os cromossomos ficam parecidos, cruzá-los gera filhos quase iguais aos pais. A mutação os empurra para outras regiões da busca.

Quanto mutar

A quantidade de genes alterados em cada filho é um parâmetro definido por quem usa o algoritmo, e o valor padrão da biblioteca é 1. O efeito desse valor depende do tamanho do cromossomo, que tem um gene para cada feature da base, mais o bias. Por isso, o que importa é a proporção de genes alterados:

Base	Features	Genes	Proporção com 1 gene mutado
Exemplo deste documento	4	5	20%
`dados_binario_1.xlsx`	18	19	5,3%
`dados_multiclasse.xlsx`	30	31	3,2%

Mutar 1 gene muda bastante um cromossomo de 5 genes, mas bem menos um de 31. Assim, um valor adequado para uma base pode ser pouco ou demais para outra.

Essa proporção equilibra aproveitar o que já foi encontrado e explorar coisas novas:

Quantidade de mutação	Vantagem	Problema
Nenhuma	Preserva tudo o que foi herdado	A busca fica limitada aos valores que já existem
Pouca	Testa novidades sem perder o que foi herdado	A busca pode ser mais lenta
Muita	Explora bastante	Destrói o que foi herdado, e a busca se aproxima de tentativas ao acaso

Testamos isso com 10 sementes, 500 gerações, fitness alvo de 1,0 e os valores padrão nos demais parâmetros, com a base `dados_binario_1.xlsx`. Os cromossomos da base têm 19 genes:

Genes mutados	Proporção	Melhor fitness no treino (média)	Acurácia no teste (média)
0	0%	0,4637	64,8%
1	5,3%	0,8123	82,6%
2	10,5%	0,8105	83,6%
4	21,1%	0,7875	81,3%
8	42,1%	0,7703	81,3%
19	100%	0,6755	77,4%

Sem mutação, a busca fica muito limitada. Com todos os genes mutados, os filhos não herdam nada dos pais, e o resultado também piora. Nesta base, os melhores valores ficaram em 1 ou 2 genes.

Por que a mutação não estraga o algoritmo

Como a mutação é aleatória, muitos filhos podem piorar, como no terceiro caso do exemplo. Isso não é um problema, porque nem todo filho entra na população. Quem decide é a próxima etapa, a atualização, que escolhe quais filhos ficam e quais cromossomos saem.

Outras formas de mutar

Nesta biblioteca, o gene é trocado por um valor novo. Essa não é a única forma de fazer mutação:

Método	Como funciona
Substituição	o gene recebe um valor novo, sem relação com o anterior
Perturbação	o gene recebe um pequeno ajuste, somado ao valor que já tinha
Por probabilidade	cada gene tem uma chance de mudar, então a quantidade varia a cada filho

> Na biblioteca, a mutação sempre sorteia valores entre -1 e 1, mesmo quando a população inicial usa outro intervalo. Os parâmetros `minimo` e `maximo` valem só para a criação da população.

Etapa 6: atualização da população

Na atualização, o algoritmo decide quem continua na população para a próxima geração. Os piores cromossomos saem, e os melhores filhos entram no lugar deles. O tamanho da população não muda. Essa é a etapa que encerra o ciclo base do programa.

Como funciona a atualização

1. Calculamos o fitness de cada filho já mutado.
2. Ordenamos a população e os filhos pelo fitness.
3. Retiramos os piores cromossomos da população.
4. Colocamos os melhores filhos nesses lugares.

A quantidade de cromossomos trocados é um parâmetro. Ela não pode ser maior que o tamanho da população nem que a quantidade de filhos.

Exemplo

Vamos usar a população da Etapa 3, supondo que o casal tenha gerado 3 filhos e que 2 cromossomos serão trocados. Os dois piores da população são C_3 e C_4, e os dois melhores filhos são F_1 e F_2. O filho F_3 é descartado (Figura 9).

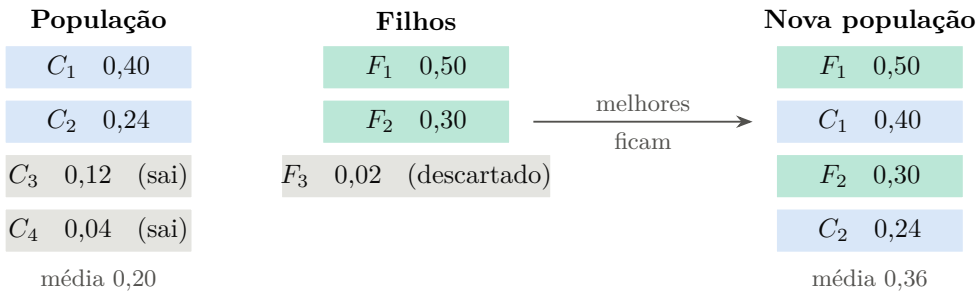


Figura 9. Os piores da população saem, e os melhores filhos entram.

O melhor fitness subiu de 0,40 para 0,50, e a média da população subiu de 0,20 para 0,36.

Quando os filhos são piores

Os melhores filhos entram mesmo que sejam piores que os cromossomos que saem. Imagine que os filhos tivessem fitness de 0,10, 0,05 e 0,02:

Antes	Depois
0,40	0,40
0,24	0,24
0,12	0,10
0,04	0,05

C_3 (0,12) foi trocado por um filho pior, e a média caiu de 0,20 para 0,1975. Isso não é um problema, porque os melhores cromossomos continuam na população, e a entrada de filhos mantém a variedade da busca.

Por que preservar os melhores

Trocar apenas os piores mantém as melhores soluções na população. Essa estratégia é chamada de *elitismo*, e impede que uma boa solução se perca por causa do acaso na seleção, no cruzamento ou na mutação.

Além disso, o algoritmo guarda uma cópia do melhor cromossomo encontrado até o momento. Assim, o resultado final nunca é perdido, mesmo que esse cromossomo saia da população em algum momento.

Quantos cromossomos trocar

Assim como na mutação, o que importa é a proporção. Os valores padrão da biblioteca são uma população de 6, com 3 filhos e 2 trocas por geração, ou seja, cerca de 33% da população:

Quantidade de trocas	Vantagem	Problema
Nenhuma	Preserva toda a população	Os filhos não entram, e o algoritmo não evolui
Poucas	Preserva as boas soluções	A população muda mais devagar
Muitas	Renova bastante a população	Boas soluções podem sair da população

Testamos isso com uma população de 6 cromossomos e 6 filhos, mantendo os demais valores padrão, em 10 sementes, 500 gerações e fitness alvo de 1,0:

Trocas por geração	Melhor fitness no treino (média)	Acurácia no teste (média)
0	0,3481	60,8%
1	0,8177	82,8%
2	0,8149	83,2%
3	0,8192	83,2%
6	0,7204	78,2%

Sem trocas, o algoritmo fica só com o melhor cromossomo da população inicial. Com 6 trocas, toda a população é substituída a cada geração, e o resultado também piora.

O fim do ciclo

Com a população atualizada, termina uma geração. O algoritmo volta para a Etapa 2 e repete o ciclo:

$$\text{fitness} \rightarrow \text{seleção} \rightarrow \text{cruzamento} \rightarrow \text{mutação} \rightarrow \text{atualização} \rightarrow \text{fitness} \rightarrow \ldots$$

O ciclo termina quando uma destas condições é atingida:

- **Fitness alvo:** o melhor cromossomo chegou ao valor definido como suficiente.
- **Limite de gerações:** a quantidade máxima de gerações foi alcançada.

A Figura 10 mostra essa evolução em um treino real, feito com a base `dados_binario_1.xlsx`.

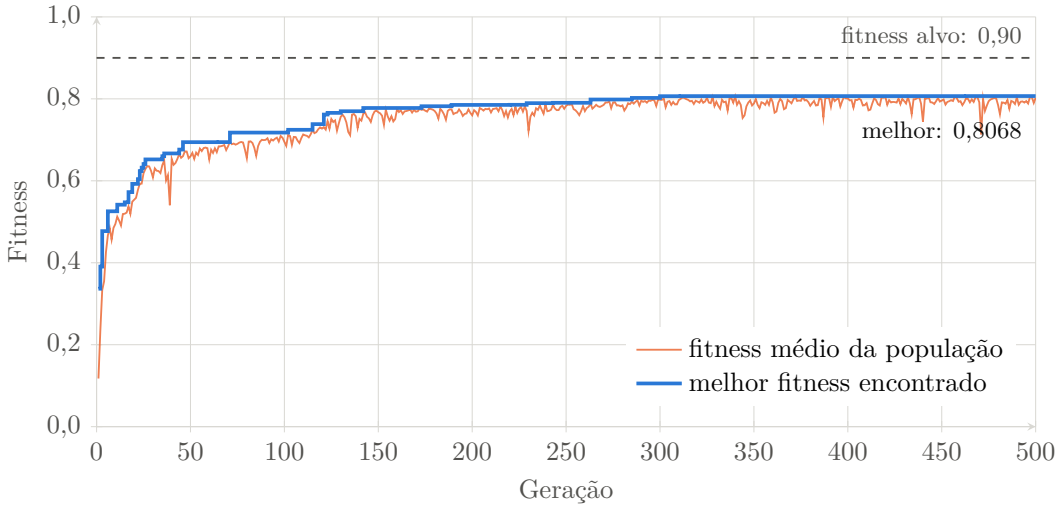


Figura 10. Evolução do fitness em 500 gerações. O alvo de 0,90 não foi atingido, então o treino parou no limite de gerações.

O resultado do treinamento é o melhor cromossomo encontrado em toda a execução, e é ele que será usado para classificar dados novos. O algoritmo genético não garante que essa seja a melhor solução possível, e sim a melhor que encontrou durante a busca. Por isso, execuções com sementes diferentes podem chegar a resultados diferentes.

Quantas gerações rodar

Mais gerações ajudam, mas o ganho diminui com o tempo. Com os valores padrão, fitness alvo de 1,0 e 10 sementes:

Gerações	Melhor fitness no treino (média)	Acurácia no teste (média)
50	0,7216	77,4%
100	0,7607	80,2%
500	0,8123	82,6%
2000	0,8180	83,3%

O efeito da semente

Como o algoritmo depende de sorteios, cada semente leva a um resultado diferente. Estes são 5 treinos com os mesmos dados e parâmetros, com 500 gerações:

Semente	Fitness no treino	Acurácia no treino	Acurácia no teste
0	0,8068	89,8%	81,5%
1	0,8195	90,5%	82,4%
2	0,8027	89,6%	83,3%
3	0,8279	91,0%	83,3%
4	0,8237	90,7%	80,6%

O maior fitness no treino não garantiu a maior acurácia no teste. A semente 2 teve o menor fitness e empatou na maior acurácia no teste. Por isso, vale testar mais de uma semente e sempre avaliar o resultado no conjunto de teste.

Parte II

Como usar a biblioteca

Os quatro passos

O uso da biblioteca tem sempre quatro passos:

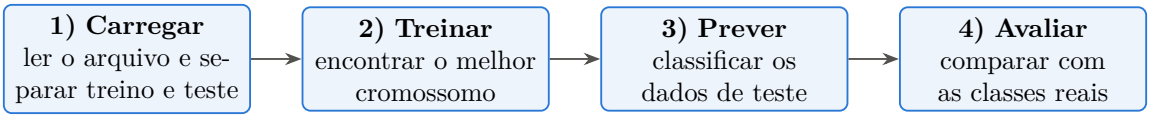


Figura 11. Os quatro passos de uso da biblioteca.

1. **Carregar os dados:** ler o arquivo e separar treino e teste.
2. **Treinar:** encontrar o melhor cromossomo usando os dados de treino.
3. **Prever:** usar esse cromossomo para classificar os dados de teste.
4. **Avaliar:** comparar as previsões com as classes reais.

O código completo cabe em poucas linhas, e cada parte é explicada a seguir:

```
from algoritmo_genetico import ler_excel, dividir_treino_teste, treinar, prever, avaliar

# 1) Carregar os dados
X, y = ler_excel("dados_binario_1.xlsx")
X_treino, X_teste, y_treino, y_teste = dividir_treino_teste(X, y, seed=0)

# 2) Treinar
modelo = treinar(X_treino, y_treino, geracoes=500, fitness_alvo=0.90, seed=0)

# 3) Prever
previsoes = prever(modelo, X_teste)

# 4) Avaliar
resultado = avaliar(y_teste, previsoes)
```

Depois desses passos, o modelo pode ser usado para classificar dados novos.

Instalação

A biblioteca requer Python 3.10 ou superior.

```
pip install algoritmo-genetico
```

O que entra e o que sai de cada função

A saída de uma função é a entrada da seguinte (Figura 12).

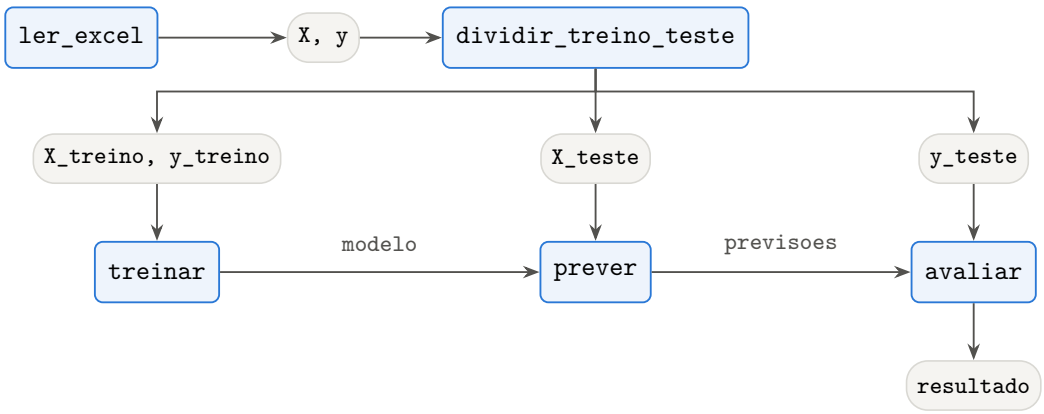


Figura 12. Como os dados passam de uma função para a outra. Funções em retângulos, dados em cápsulas.

Função	Recebe	Retorna	Para que serve a saída
`ler_excel` / `ler_csv`	caminho do arquivo	`X, y`	entram no `dividir_treino_teste`
`ler_excel` / `ler_csv` com `tem_classe=False`	caminho do arquivo	`X`	entra no `prever`
`dividir_treino_teste`	`X, y`	`X_treino`, `X_teste`, `y_treino`, `y_teste`	`X_treino` e `y_treino` entram no `treinar`, `X_teste` entra no `prever`, e `y_teste` entra no `avaliar`
`treinar`	`X_treino`, `y_treino`	`modelo` (dicionário)	entra no `prever`
`prever`	`modelo`, `X_teste`	`previsoes` (array)	entram no `avaliar`
`avaliar`	`y_teste`, `previsoes`	`resultado` (dicionário)	fim do processo: mostra como o modelo se saiu
`matriz_confusao`	`y_teste`, `previsoes`	`matriz` (array)	opcional: só a matriz, sem o resto da avaliação

Dois cuidados com a ordem:

- O `prever` recebe primeiro o `modelo` e depois as features.
- O `avaliar` recebe primeiro as classes reais e depois as previstas.

1) Carregar os dados

Formato do arquivo

O arquivo, em `.xlsx` ou `.csv`, deve seguir este layout:

Lições	X1	X2	...	Xn	CLASS
Y001	1	0	...	1	1
Y002	0	1	...	0	0

- **Primeira coluna:** identifica a lição e é ignorada.
- **Colunas do meio:** são as features. Precisam ser números, sem células vazias.
- **Última coluna:** é a classe.

Ler o arquivo

```
X, y = ler_excel("dados_binario_1.xlsx")
```

Para arquivos `.csv`, use `ler_csv` da mesma forma.

O que retorna: dois arrays, sempre nesta ordem.

Variável	O que é	Formato no exemplo
`X`	features de todas as lições	539 linhas e 18 colunas
`y`	classe de cada lição	539 valores

Separar treino e teste

```
X_treino, X_teste, y_treino, y_teste = dividir_treino_teste(X, y, seed=0)
```

O modelo aprende com o treino, e o teste fica guardado para verificar se ele acerta lições que nunca viu. Por padrão, 80% das lições vão para o treino, e a proporção de cada classe é mantida nos dois conjuntos. O parâmetro `seed` faz a divisão sair igual em toda execução.

O que retorna: quatro arrays, sempre nesta ordem.

Posição	Variável	O que é	Formato no exemplo
1º	`X_treino`	features do treino	431 linhas e 18 colunas
2º	`X_teste`	features do teste	108 linhas e 18 colunas
3º	`y_treino`	classes do treino	431 valores
4º	`y_teste`	classes do teste	108 valores

> A ordem é primeiro os dois `X`, depois os dois `y`, com treino antes de teste em cada par. A função devolve os arrays por posição, então os nomes das variáveis à esquerda precisam seguir essa ordem. Copie a linha acima exatamente como está.

2) Treinar

```
modelo = treinar(X_treino, y_treino, geracoes=500, fitness_alvo=0.90, seed=0)
```

O `treinar` executa o algoritmo genético e recebe apenas os dados de treino. O treino para quando o melhor cromossomo atinge o `fitness_alvo` ou quando acabam as gerações.

A cada geração, ele mostra o progresso na tela. Esta é a última linha do exemplo:

```
Geração  500 | melhor fitness: 0.8068 | acurácia: 89.79%
```

Para treinar sem essas mensagens, use `verbose=False`.

O que retorna: um único dicionário, aqui chamado de `modelo`. Cada informação é acessada pelo nome:

Chave	O que é	Valor no exemplo
`modelo["cromossomo"]`	genes do melhor cromossomo: o bias e um peso para cada feature	array com 19 valores
`modelo["fitness"]`	fitness desse cromossomo no treino	0,8068
`modelo["acuracia"]`	acurácia desse cromossomo no treino	0,8979
`modelo["geracao"]`	geração em que o treino parou	500
`modelo["thresholds"]`	pontos de corte usados	`None` (duas classes)
`modelo["historico"]`	evolução do treino, em três listas	500 valores em cada lista

As três listas do histórico são acessadas assim:

```
modelo["historico"]["melhor"]    # melhor fitness até cada geração
modelo["historico"]["medio"]     # fitness médio da população em cada geração
modelo["historico"]["acuracia"]  # acurácia do melhor cromossomo até cada geração
```

3) Prever

```
previsoes = prever(modelo, X_teste)
```

O `prever` recebe o dicionário gerado no treino e as features das lições a classificar. As classes reais não são passadas, porque é isso que o modelo precisa descobrir.

O que retorna: um array com a classe prevista para cada lição, na mesma ordem das linhas recebidas. No exemplo, são 108 valores, cada um 0 ou 1.

O que acontece por dentro

Depois do treino, o algoritmo genético não roda mais. O modelo é só o melhor cromossomo, e prever é fazer a mesma conta da Etapa 2 com ele.

Imagine um modelo treinado para dizer se um cliente é adimplente (classe 1) ou inadimplente (classe 0), com 4 features, nesta ordem:

Feature	Significado
X1	tem renda alta?
X2	tem emprego fixo?
X3	já atrasou um pagamento?
X4	tem outra dívida?

Supondo que o melhor cromossomo seja $[1,\ 2,\ 1,\ -3,\ -1]$:

Cliente	Cálculo	Nota	Classe	Resultado
$[1,\ 1,\ 0,\ 1]$	$1 + 2 \cdot 1 + 1 \cdot 1 - 3 \cdot 0 - 1 \cdot 1$	3	1	adimplente
$[0,\ 1,\ 1,\ 1]$	$1 + 2 \cdot 0 + 1 \cdot 1 - 3 \cdot 1 - 1 \cdot 1$	-2	0	inadimplente

A resposta é uma previsão com base nos padrões da base de treino, não uma certeza. O significado de cada classe vem dos dados usados no treino.

Quantas lições o prever aceita

O que recebe	Formato	O que retorna
Várias lições	matriz com uma lição por linha	array com uma classe por lição
Uma lição	vetor com um valor por feature	array com uma única classe

4) Avaliar

Agora comparamos as previsões com as classes reais do teste, que o modelo não viu durante o treino.

```
resultado = avaliar(y_teste, previsoes)
```

Passe primeiro as classes reais e depois as previstas. A função mostra este resumo na tela:

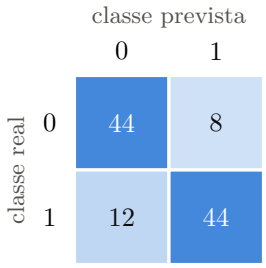
```
acurácia: 81.48%
fitness: 0.6648

recall por classe:
  0: 44/52 = 84.62%
  1: 44/56 = 78.57%

matriz de confusão:
real \ previu    0    1
            0   44    8
            1   12   44
```

Cada parte responde a uma pergunta:

- **Acurácia:** de todas as lições, quantas o modelo acertou?
- **Fitness:** o modelo foi bem em todas as classes? É o produto dos recalls.
- **Recall por classe:** das lições de cada classe, quantas ele acertou?
- **Matriz de confusão:** com o que ele confundiu cada classe? Cada linha é a classe real, cada coluna é a classe prevista, e a diagonal mostra os acertos (Figura 13).



Das 52 lições da classe 0, o modelo acertou 44 e errou 8.

Das 56 lições da classe 1, acertou 44 e errou 12.

Quanto mais escura a célula, mais lições caíram nela. Em um bom modelo, os valores se concentram na diagonal.

Figura 13. A matriz de confusão do exemplo com duas classes.

No exemplo, a acurácia foi de 89,79% no treino e 81,48% no teste. É comum o resultado no teste ser menor, e é ele que indica como o modelo deve se sair com dados novos.

O que retorna: um dicionário com os mesmos valores. Para receber apenas o dicionário, sem mostrar o resumo, use `verbose=False`.

Chave	O que é	Valor no exemplo
`resultado["acuracia"]`	fração de lições que o modelo acertou	0,8148
`resultado["fitness"]`	produto dos recalls	0,6648
`resultado["recall"]`	array com o recall de cada classe, da classe 0 em diante	[0,8462; 0,7857]
`resultado["matriz"]`	array com a matriz de confusão	[[44, 8], [12, 44]]

Se uma classe não aparecer nos dados avaliados, o recall dela não pode ser calculado. Nesse caso, o resumo mostra "sem amostras", e o recall dessa classe e o fitness ficam como `nan`.

Para ver somente a matriz de confusão, importe e use `matriz_confusao(y_teste, previsoes)`.

De onde vêm as medidas

As medidas saem diretamente da matriz de confusão. Com os valores deste exemplo:

Medida	Como calcular	Valor
Acurácia	soma da diagonal dividida pelo total: $(44 + 44)/108$	0,8148
Recall da classe 0	acertos dividido pela linha da classe: $44/52$	0,8462
Recall da classe 1	acertos dividido pela linha da classe: $44/56$	0,7857
Fitness	produto dos recalls: $0{,}8462 \times 0{,}7857$	0,6648

Para usar um valor depois, acesse o dicionário pelo nome:

```
print(resultado["acuracia"])   # 0.8148148148148148
print(resultado["matriz"][1, 0])   # classe 1 prevista como classe 0: 12
```

Classificar dados novos

Depois de treinar e avaliar o modelo, ele pode ser usado com dados que ainda não têm classe, como clientes novos. Como não há classe real para comparar, o `avaliar` não se aplica aqui. A qualidade esperada do modelo é a que foi medida no teste.

Vários clientes de uma planilha

O arquivo segue o mesmo layout, mas sem a coluna de classe:

Cliente	X1	X2	...	Xn
Ana	1	0	...	1
Bruno	0	1	...	0

As features precisam estar na mesma ordem do treino. A biblioteca confere a quantidade de colunas, mas não a ordem, então uma troca de colunas gera previsões erradas sem aviso.

Use `tem_classe=False` para avisar que o arquivo não tem a coluna de classe:

```
X_novo = ler_excel("clientes_novos.xlsx", tem_classe=False)
previsoes = prever(modelo, X_novo)
```

O que retorna: com `tem_classe=False`, o `ler_excel` devolve só o `X`, e não `X, y`. O `prever` devolve um array com a classe de cada linha, na mesma ordem do arquivo.

As features não podem ter células vazias. Se tiverem, o `prever` mostra um erro em vez de classificar.

Para gerar uma planilha com a classe de cada cliente, use o pandas, que é instalado junto com a biblioteca:

```
import pandas as pd

df = pd.read_excel("clientes_novos.xlsx")
df["CLASSE_PREVISTA"] = previsoes
df.to_excel("clientes_classificados.xlsx", index=False)
```

Um único cliente

Os dados também podem ser escritos diretamente, com um valor para cada feature, na mesma ordem do treino. O exemplo abaixo supõe um modelo treinado com 4 features:

```
import numpy as np

cliente = np.array([1, 0, 0, 1])
classe = prever(modelo, cliente)[0]
```

O resultado do `prever` continua sendo um array, agora com um único valor, então usamos `[0]` para obter a classe.

Guardar o modelo

O `modelo` só existe enquanto o programa está aberto. Para usá-lo depois sem treinar de novo, salve o dicionário com o `pickle`, do próprio Python:

```
import pickle

# depois de treinar
with open("modelo.pkl", "wb") as f:
    pickle.dump(modelo, f)

# em outra execução
with open("modelo.pkl", "rb") as f:
    modelo = pickle.load(f)
```

> Só carregue arquivos `.pkl` de fontes confiáveis, porque abrir um pode executar código.

O caminho completo

Juntando tudo, um programa treina e guarda o modelo, e outro o carrega para classificar dados novos:

```
import pickle
from algoritmo_genetico import ler_excel, dividir_treino_teste, treinar, prever, avaliar

# treinar, avaliar e guardar
X, y = ler_excel("dados_binario_1.xlsx")
X_treino, X_teste, y_treino, y_teste = dividir_treino_teste(X, y, seed=0)
modelo = treinar(X_treino, y_treino, geracoes=500, seed=0, verbose=False)
resultado = avaliar(y_teste, prever(modelo, X_teste))

with open("modelo.pkl", "wb") as f:
    pickle.dump(modelo, f)

# em outra execução: carregar e classificar
with open("modelo.pkl", "rb") as f:
    modelo = pickle.load(f)

X_novo = ler_excel("clientes_novos.xlsx", tem_classe=False)
previsoes = prever(modelo, X_novo)
```

Com mais de duas classes

Os quatro passos são os mesmos, e o modelo passa a ser ordinal. Só mudam os passos 1 e 2, e o passo 4 ganha uma opção.

No passo 1, as classes precisam ser números de 0 a $K-1$, da menor para a maior. Se estiverem em texto, converta-as depois de ler o arquivo. A ordem abaixo foi encontrada pelo notebook multiclasse testando as combinações, e não é uma ordem conhecida:

```
import numpy as np

X, y_letras = ler_excel("dados_multiclasse.xlsx")

ordem = ["P", "D", "G", "M", "S"]
y = np.array([ordem.index(letra) for letra in y_letras])

X_treino, X_teste, y_treino, y_teste = dividir_treino_teste(X, y, seed=0)
```

No passo 2, passe $K-1$ thresholds em ordem crescente. Com 5 classes, são 4 cortes:

```
thresholds = np.array([-1.5, -0.5, 0.5, 1.5])

modelo = treinar(X_treino, y_treino, thresholds=thresholds,
                 geracoes=2000, fitness_alvo=1.0, seed=0)
```

O passo 3 não muda:

```
previsoes = prever(modelo, X_teste)
```

No passo 4, `nomes_classes` troca os números pelos nomes no resumo:

```
resultado = avaliar(y_teste, previsoes, nomes_classes=ordem)
```

```
acurácia: 75.00%
fitness: 0.2380

recall por classe:
  P: 6/13 = 46.15%
  D: 7/7 = 100.00%
  G: 9/12 = 75.00%
  M: 6/8 = 75.00%
  S: 11/12 = 91.67%
```

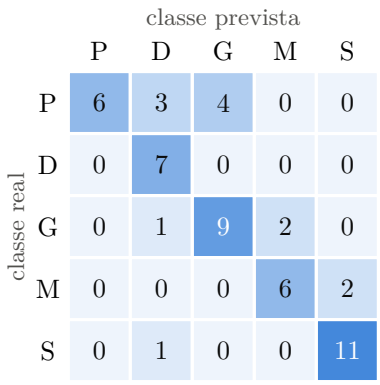


Figura 14. A matriz de confusão do exemplo com 5 classes.

A acurácia de 75% esconde uma diferença entre as classes: o modelo acerta todas as lições de D, mas menos da metade das de P. O recall por classe mostra isso.

O `prever` devolve números. Para classificar dados novos e obter os nomes das classes, converta o resultado:

```
nomes = [ordem[classe] for classe in prever(modelo, X_novo)]
```

Como escolher os thresholds

Os thresholds são definidos por quem usa a biblioteca e ficam fixos durante o treino. Quem se adapta são os genes, que precisam gerar notas dentro das faixas certas. Por isso, cortes igualmente espaçados, como os do exemplo, são um bom ponto de partida.

Como escolher a ordem das classes

Quando a ordem real das classes é conhecida, basta usá-la. Quando não é, o notebook do exemplo multiclasse testa as combinações possíveis. Com 5 classes, são 120 ordens, mas uma ordem e a sua inversa dão o mesmo resultado, então bastam 60:

Ordem testada	Fitness médio no treino
P < D < G < M < S (a melhor)	0,3737
D < P < G < M < S (a segunda melhor)	0,3678
G < P < S < D < M (a pior)	0,0518

As melhores ordens ficam com fitness parecido, e a vencedora pode mudar com outras sementes. Já a diferença para as piores é grande. A escolha usa só os dados de treino, para o teste continuar sendo inédito para o modelo.

Parâmetros do `treinar`

Parâmetro	Padrão	Descrição
`X`	obrigatório	matriz com as features
`y`	obrigatório	vetor com a classe de cada lição
`qtd_cromossomos`	`6`	tamanho da população
`qtd_genes_mutados`	`1`	quantos genes mudam em cada filho
`qtd_filhos`	`3`	quantos filhos são gerados por geração
`qtd_substituicoes`	`2`	quantos piores cromossomos são trocados pelos melhores filhos
`geracoes`	`100`	número máximo de gerações
`fitness_alvo`	`0.90`	fitness em que o treino para
`thresholds`	`None`	`None` para duas classes, ou $K-1$ cortes para K classes
`minimo`	`-1.0`	menor valor dos genes na população inicial
`maximo`	`1.0`	maior valor dos genes na população inicial
`seed`	`None`	semente para repetir o mesmo resultado
`verbose`	`True`	mostra o progresso a cada geração

- `qtd_substituicoes` não pode ser maior que `qtd_cromossomos` nem que `qtd_filhos`.
- `qtd_genes_mutados` não pode ser maior que a quantidade de genes, que é a de features mais o bias.
- `minimo` e `maximo` valem só para a população inicial. A mutação sempre sorteia valores entre -1 e 1.

Por onde começar a ajustar

Nos testes das etapas anteriores, com a base `dados_binario_1.xlsx`, os parâmetros que mais mudaram o resultado foram estes:

- **`geracoes`:** a média do melhor fitness subiu de 0,72 com 50 gerações para 0,81 com 500.
- **`qtd_genes_mutados`:** os melhores resultados vieram com 1 ou 2 genes, e valores altos pioraram o fitness.
- **`qtd_substituicoes`:** trocar nenhum cromossomo ou a população inteira piorou o resultado.

Já `qtd_cromossomos` e `qtd_filhos` fizeram pouca diferença nesses testes. Esses resultados valem para essa base e podem mudar em outras.

Usando cada etapa separadamente

As etapas do algoritmo também podem ser chamadas uma a uma, para estudar ou testar cada parte. O notebook `exemplo_binario.ipynb` mostra essa sequência passo a passo, com as saídas de cada etapa. Elas ficam em módulos próprios:

```
from algoritmo_genetico.criacao import criar_cromossomos
from algoritmo_genetico.fitness import fitness_binario, fitness_ordinal, fitness_percentual
from algoritmo_genetico.selecao import selecionar_pais_roleta
from algoritmo_genetico.cruzamento import cruzar_um_ponto
from algoritmo_genetico.mutacao import mutar
from algoritmo_genetico.atualizacao import atualizar_populacao
```

Uma geração completa é esta sequência, em que cada linha usa o que as anteriores produziram:

```
populacao = criar_cromossomos(qtd_cromossomos=6, qtd_genes=X.shape[1] + 1)
fitnesses = fitness_binario(populacao, X, y)
percentuais = fitness_percentual(fitnesses)
pai, mae = selecionar_pais_roleta(populacao, percentuais)
filhos = cruzar_um_ponto(pai, mae, qtd_filhos=3)
filhos = mutar(filhos, qtd_genes_mutados=1)
fitnesses_filhos = fitness_binario(filhos, X, y)
populacao = atualizar_populacao(populacao, fitnesses, filhos, fitnesses_filhos,
                                qtd_substituicoes=2)
```

A nova `populacao` volta para o cálculo do fitness, e o ciclo recomeça. Os formatos abaixo usam o exemplo de 18 features, com uma população de 6 cromossomos e 3 filhos:

Função	Recebe	Retorna	Formato no exemplo
`criar_cromossomos`	quantidade de cromossomos e de genes	`populacao`: matriz em que cada linha é um cromossomo	6 linhas e 19 colunas
`fitness_binario`	`populacao, X, y`	`fitnesses`: fitness de cada cromossomo	6 valores
`fitness_ordinal`	`populacao, X, y, thresholds`	fitness de cada cromossomo, para várias classes	um valor por cromossomo
`fitness_percentual`	`fitnesses`	`percentuais`: chance de cada cromossomo na roleta	6 valores que somam 1
`selecionar_pais_roleta`	`populacao, percentuais`	`pai, mae`: dois cromossomos	19 valores cada
`cruzar_um_ponto`	`pai, mae`, quantidade de filhos	`filhos`: matriz em que cada linha é um filho	3 linhas e 19 colunas
`mutar`	`filhos`, quantidade de genes mutados	`filhos` mutados, sem alterar o array original	3 linhas e 19 colunas
`atualizar_populacao`	`populacao, fitnesses, filhos, fitnesses_filhos`, quantidade de trocas	nova `populacao`	6 linhas e 19 colunas

Na linha de `fitnesses_filhos`, a mesma função `fitness_binario` calcula o fitness dos filhos, com 3 valores no exemplo. No caso com várias classes, use `fitness_ordinal` nas duas chamadas.

Erros comuns

As funções verificam o que recebem e explicam o problema na mensagem de erro. Estas são as situações mais comuns:

Situação	Mensagem	O que fazer
Célula vazia nas features	*X tem células vazias ou valores inválidos (NaN ou infinito), corrija esses dados antes de continuar*	preencher ou remover as células
Classes em texto, com thresholds	*y deve conter inteiros de 0 a K-1, classes em texto precisam ser convertidas antes de treinar*	converter as classes para números
Classes diferentes de 0 e 1, sem thresholds	*y deve conter apenas 0 e 1, encontrou [0 1 2]*	passar `thresholds` para usar várias classes
Thresholds a menos para as classes	*y deve conter inteiros entre 0 e 1, encontrou [0 1 2]*	passar $K - 1$ cortes
Thresholds a mais para as classes	*y deve ter pelo menos uma amostra de cada classe. Classes sem amostras: [3]*	passar $K - 1$ cortes
Planilha sem classe lida sem `tem_classe=False`	*X tem 17 features, mas o cromossomo foi treinado com 18 features*	usar `tem_classe=False`
Mais trocas que filhos	*qtd_substituicoes deve estar entre 0 e 1 (menor valor entre qtd_cromossomos e qtd_filhos), recebeu 2*	reduzir as trocas ou aumentar os filhos

> Se as colunas estiverem fora da ordem do treino, nenhum erro aparece, porque a biblioteca só confere a quantidade de colunas. Essa verificação fica por conta de quem usa.

Encontrou um erro que não está aqui, ou tem uma sugestão? Abra uma issue no repositório do projeto ou fale com a gente pelos contatos da seção Sobre este projeto.